Deep RAPM: A Set Transformer for OnCourt Contribution

Aaron Danielson

March 18, 2026

Abstract

Regularized Adjusted Plus-Minus (RAPM) is a cornerstone of modern NBA player evaluation, but its linear, additive structure limits its ability to capture the richness of basketball: intra-lineup synergy, offense-defense matchup effects, and nonlinear interactions between players. We propose *Deep RAPM*, a transformer-based architecture that replaces each player's scalar offensive and defensive coefficient with a learned latent embedding. Offensive and defensive lineups are treated as unordered sets and encoded jointly via a Set Transformer with role-position attention biases. The resulting 10-player self-attention mechanism produces a structured 10×10 attention matrix whose quadrants correspond to offense-offense synergy, offense-defense matchups, defense-offense reactions, and defense-defense coordination. The model is trained to predict possession-level scoring outcomes and inherits counterfactual lineup evaluation as a natural byproduct of the embedding framework. We describe the full architecture, training procedure, and interpretability properties.

1 Introduction

Regularized Adjusted Plus-Minus (RAPM) [1] has become the workhorse of NBA player evaluation. Given the ten players on the court during a possession, RAPM estimates each player's marginal contribution by regressing a possession-level outcome onto a $2N$-dimensional indicator vector: one entry per player on offense, one per player on defense. A ridge penalty handles the sparsity of lineup observations and shrinks estimates toward zero. The result is a pair of scalar coefficients per player — one offensive, one defensive — that quantify additive contribution independent of teammates and opponents.

This simplicity is also a limitation. The model assumes that the offensive quality of a lineup is exactly the sum of five individual offensive ratings, and similarly for defense. Three concrete deficiencies follow:

1. **Intra-lineup synergy.** Two shooters who stretch the floor together create spacing that neither generates alone. A pair of bigs who switch well on defense are more than the sum of their individual defensive ratings. These interaction effects are invisible to an additive model.

2. **Matchup effects.** How a defensive lineup performs depends on *who* they are guarding. A rim protector's value varies dramatically against a drive-heavy offense versus a post-heavy one. Cross-team interactions are structurally absent from RAPM.

3. **Temporal dynamics.** RAPM treats each possession as i.i.d. Momentum, fatigue, and in-game run dynamics are ignored except insofar as recency weighting is applied.

We propose *Deep RAPM*, which addresses the first two deficiencies directly by replacing scalar coefficients with learned latent embeddings and modeling the full ten-player context via a Set Transformer. The third deficiency — temporal dynamics — is a natural extension we leave to future work.

2 Problem Setup

We model possession-level outcomes. Each lineup is represented as an ordered 5-tuple for computation, but the model is required to be permutation-invariant to that ordering — so it effectively treats each lineup as an unordered set. For possession i, let:

- $\mathbf{o}_i = (o_{i,1}, \ldots, o_{i,5})$ be the indices of the five **offensive** players on the court (in an arbitrary fixed order),
- $\mathbf{x}_i = (x_{i,1}, \ldots, x_{i,5})$ be the indices of the five **defensive** players (in an arbitrary fixed order),
- $\mathbf{g}_i \in \mathbb{R}^p$ be a vector of **gamestate features** (e.g., score differential, time remaining, home/away indicator),
- y_i be the outcome of interest.

The outcome y_i can take several forms: the number of points scored during a possession (an integer in $\{0, 1, 2, 3, 4\}$), a binary indicator (e.g., whether the possession results in an assist), or a bounded real (e.g., possession length). We focus on point scoring throughout.

The general prediction rule is

$$\hat{y}_i = f_\theta\big(\mathbf{o}_i, \mathbf{x}_i, \mathbf{g}_i\big).$$

In classical RAPM, f_θ is a linear function of a $2N$-dimensional indicator vector; the parameter vector θ is a ridge-regularized least squares solution. We retain this general form but relax both the linearity and the scalar-per-player constraints.

Permutation invariance. A crucial structural constraint: the five offensive players have no natural ordering, and neither do the five defensive players. Any model that depends on the ordering of players within a lineup is modeling an artifact of data construction rather than basketball. We therefore require f_θ to be *permutation-invariant* in $\mathbf{o}_i$ and separately permutation-invariant in $\mathbf{x}_i$.

3 From Scalar Coefficients to Latent Embeddings

The central generalization is to replace each player's pair of scalar RAPM coefficients with a pair of latent embedding vectors. Specifically, for each player $j \in \{1, \ldots, N\}$ we learn:

$$u_j^o \in \mathbb{R}^d, \qquad u_j^d \in \mathbb{R}^d,$$

where u_j^o is the player's *offensive embedding* and u_j^d is the *defensive embedding*. Classical RAPM is recovered in the special case $d = 1$ with a linear prediction head and no interaction across players.

Conceptually, the model assigns distinct offensive and defensive latent roles to each player; operationally, this is implemented through a shared player embedding table composed with role

embeddings and a projection MLP (Section 4.1), so both roles emerge from a single learned representation rather than two independent tables.

For $d > 1$, these embeddings encode richer player characteristics. The offensive embedding for a player can encode shooting tendency, pick-and-roll usage, finishing ability, and more — as implicit features learned from possession outcomes, without hand-crafting any of these statistics. The embeddings are trained end-to-end together with all other model parameters.

The prediction rule becomes:

$$h_i^o = \text{Encode}_\theta\big(\{u^o_{o_{i,j}}\}_{j=1}^5\big) \in \mathbb{R}^d, \tag{1}$$

$$h_i^d = \text{Encode}_\theta\big(\{u^d_{x_{i,k}}\}_{k=1}^5\big) \in \mathbb{R}^d, \tag{2}$$

$$\hat{y}_i = \text{MLP}_\theta\big([h_i^o;\, h_i^d;\, \mathbf{g}_i]\big), \tag{3}$$

where Encode_θ is a permutation-invariant set encoder and $[\,;\,]$ denotes concatenation.

From Deep Sets to Set Transformers. The simplest permutation-invariant encoder is a *Deep Sets* aggregator [2]: apply a shared MLP ϕ_θ to each embedding, sum, then apply another MLP ρ_θ:

$$h_i^o = \rho_\theta\Big(\tfrac{1}{5}\sum_{j=1}^5 \phi_\theta(u^o_{o_{i,j}})\Big).$$

Deep Sets handles permutation invariance and captures nonlinear player-level transformations, but it aggregates via a symmetric function and cannot model *interactions between players within a lineup*. Two shooters whose value depends on their co-presence remain unrepresented.

A *Set Transformer* [3] addresses this by applying multihead self-attention over the set before pooling. Each element attends to every other element, so the aggregated representation of a player is contextual — it depends on who else is in the lineup. This is the architecture we adopt.

Crucially, we encode the offensive and defensive lineups *jointly* in a single self-attention step over all ten players. This allows the model to learn offense-defense interaction effects alongside intra-team synergy, without requiring separate cross-attention modules.

4 The DeepRAPM Architecture

4.1 Input Embeddings

Each of the ten players in possession i is characterized by three categorical inputs:

- $p \in \{1, \ldots, N\}$: the player's identity,
- $r \in \{0, 1\}$: the player's role (0 = offense, 1 = defense),
- $s \in \{0, \ldots, 4\}$: the player's discrete position index (PG, SG, SF, PF, C).

We learn separate embedding tables for each:

$$\mathbf{e} = \text{PlayerEmbed}(p) \in \mathbb{R}^d,$$
$$\mathbf{r} = \text{RoleEmbed}(r) \in \mathbb{R}^d.$$

The initial player token is formed by concatenating and projecting through an MLP:

$$\mathbf{z}_i^{(0)} = \mathrm{MLP}\big([\mathbf{e}_i;\, \mathbf{r}_i]\big) \in \mathbb{R}^d.$$

This MLP imbues each token with an offensive or defensive interpretation before any cross-player attention occurs. Position s is not included in the initial embedding; instead, it enters as a structural prior via the attention bias described below.

We stack the ten tokens into a matrix:

$$\mathbf{Z} = \big[\mathbf{z}_1^{(0)}, \ldots, \mathbf{z}_{10}^{(0)}\big]^\top \in \mathbb{R}^{10\times d},$$

with the five offensive players occupying rows 1–5 and the five defensive players occupying rows 6–10.

4.2 Role-Position Attention Bias

Standard self-attention treats all pairs of tokens symmetrically beyond their embedding similarity. In basketball, however, relational structure between players is strongly patterned by role and position: point guards tend to guard point guards, rim protectors are most relevant as defensive targets for slashers, and so on. We encode these priors as a *learned attention bias*.

For each player i, define the combined role-position index:

$$c_i = r_i \cdot K + s_i, \quad K = 5,$$

so $c_i \in \{0, \ldots, 9\}$ indexes one of ten role-position categories (5 offensive positions, 5 defensive positions). We learn a bias tensor:

$$\boldsymbol{B} \in \mathbb{R}^{H\times 10\times 10},$$

where H is the number of attention heads. The bias between player i and player j in head h is:

$$\mathrm{Bias}_{h,i,j} = \boldsymbol{B}_{h,\,c_i,\,c_j}.$$

This bias is added to the raw attention logit before the softmax. A large positive value encourages attention between a particular pair of role-position categories; a large negative value suppresses it. These biases are learned from data, not hand-specified — but they provide an inductive bias that the model can exploit early in training and refine as it observes more possessions.

4.3 Self-Attention Mechanism

We apply standard multihead self-attention over the full $10 \times d$ token matrix $\mathbf{Z}$. For head h, define:

$$\begin{aligned}
\mathbf{Q}_h &= \mathbf{Z}\mathbf{W}_h^Q \in \mathbb{R}^{10\times d'_h},\\
\mathbf{K}_h &= \mathbf{Z}\mathbf{W}_h^K \in \mathbb{R}^{10\times d'_h},\\
\mathbf{V}_h &= \mathbf{Z}\mathbf{W}_h^V \in \mathbb{R}^{10\times d'_h},
\end{aligned}$$

where $d'_h = d'/H$ and d' is the internal dimension. The attention score between player i (query) and player j (key) is:

$$\alpha_{ij}^{(h)} = \frac{\mathbf{q}_i^{(h)} \cdot \mathbf{k}_j^{(h)}}{\sqrt{d'_h}} + \boldsymbol{B}_{h,\,c_i,\,c_j}.$$

The row-wise softmax gives each player's attention distribution over the full ten-player context:

$$\text{softmax}\big(\boldsymbol{A}^{(h)}\big)_{ij} = \frac{\exp(\alpha_{ij}^{(h)})}{\sum_{j'=1}^{10} \exp(\alpha_{ij'}^{(h)})}.$$

The attention output for player i in head h is:

$$\text{Attn}_i^{(h)} = \sum_{j=1}^{10} \text{softmax}\big(\boldsymbol{A}^{(h)}\big)_{ij} \cdot \mathbf{v}_j^{(h)}.$$

Intuitively, row i of the attention matrix is the distribution of player i's "attention" over all players on the court. An offensive player attending strongly to a specific defensive player reflects a learned matchup relevance; an offensive player attending to teammates reflects a learned synergy signal.

4.4 The 10×10 Attention Matrix and Its Structure

The full attention score matrix $\boldsymbol{A}^{(h)} \in \mathbb{R}^{10\times 10}$ has a natural block structure corresponding to the four types of player-player interaction:

$$\boldsymbol{A}^{(h)} = \begin{bmatrix} \text{OO}_{5\times 5} & \text{OD}_{5\times 5} \\ \text{DO}_{5\times 5} & \text{DD}_{5\times 5} \end{bmatrix}$$

- OO: Offense-to-Offense. Captures within-lineup synergy on offense — which offensive players amplify each other's embeddings.
- OD: Offense-to-Defense. Captures targeting matchups — how offensive players attend to (i.e., are contextualized by) specific defenders.
- DO: Defense-to-Offense. Captures defensive reactions — how defenders contextualize their representations based on specific offensive threats.
- DD: Defense-to-Defense. Captures defensive coordination — how defenders in a lineup shape each other's representations.

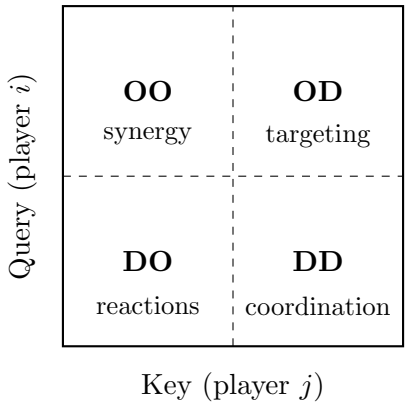


Figure 1: Block structure of the 10×10 attention matrix in DeepRAPM. Each entry α_{ij} reflects how much player i's updated representation draws on player j's value vector, modulated by learned role-position biases.

By applying the learned bias $\boldsymbol{B}$, the model can place different structural priors on each quadrant. For example, it may learn to largely suppress OO cross-diagonal entries (if players at unlike offensive positions rarely interact) while amplifying specific OD entries corresponding to primary matchup pairs.

4.5 Multihead Aggregation and Transformer Layer

The outputs of H attention heads are concatenated and projected back to $\mathbb{R}^d$:

$$\text{MultiHead}(\mathbf{Z}) = \text{Concat}(\text{Attn}_1, \ldots, \text{Attn}_H) \cdot \mathbf{W}^O,$$

where $\mathbf{W}^O \in \mathbb{R}^{(H \cdot d'_h) \times d}$. Each attention head can specialize in a different type of interaction — one head might capture positional matchups, another team-level synergy patterns.

A full transformer layer applies pre-norm, residual connections, and a position-wise feedforward network:

$$\begin{aligned}
\mathbf{Z}_1 &= \text{LayerNorm}(\mathbf{Z}), \\
\mathbf{Z}_2 &= \mathbf{Z} + \text{MultiHead}(\mathbf{Z}_1), \\
\mathbf{Z}_3 &= \text{LayerNorm}(\mathbf{Z}_2), \\
\mathbf{Z}_4 &= \mathbf{Z}_2 + \text{MLP}(\mathbf{Z}_3).
\end{aligned}$$

We stack L such layers to obtain contextual player representations $\mathbf{Z}^{(L)} \in \mathbb{R}^{10 \times d}$.

4.6 Pooling and Prediction Head

After L transformer layers, we obtain a contextually enriched representation for each of the ten players. We pool these into a single lineup embedding via mean pooling:

$$\mathbf{z}_{\text{lineup}} = \frac{1}{10} \sum_{i=1}^{10} \mathbf{z}_i^{(L)} \in \mathbb{R}^d.$$

Mean pooling is permutation-invariant by construction: the model's prediction does not depend on the order in which offensive or defensive players are indexed.

The lineup embedding is concatenated with the gamestate vector $\mathbf{g} \in \mathbb{R}^p$ and passed to a prediction head:

$$\hat{y} = \text{MLP}_{\text{head}}\big([\mathbf{z}_{\text{lineup}}; \mathbf{g}]\big).$$

5 Theoretical Properties

We establish four properties that collectively position Deep RAPM as a principled generalization of the classical RAPM framework: backward compatibility (RAPM is a special case), strict expressivity (Deep RAPM represents functions RAPM cannot), structural correctness (permutation invariance is guaranteed), and a regularization bridge (the ridge penalty is the boundary case). We close with a discussion of identifiability and the formal player-value estimand that connects these results to player evaluation.

5.1 RAPM as a Special Case

Theorem 5.1 (Backward compatibility)**.** *Classical RAPM is a special case of Deep RAPM. Specifically, there exist parameter settings under which Deep RAPM produces predictions identical to the RAPM linear model:*

$$\hat{y}_i = \sum_{j \in \mathbf{o}_i} \beta_j^o + \sum_{k \in \mathbf{x}_i} \beta_k^d.$$

Proof. We construct an explicit parameter setting. Set the model dimension to $d = 1$, transformer depth to $L = 0$ (no attention), and retain the architecture's mean pooling.

Player and role embeddings. We use an internal player embedding dimension of 2 prior to the projection MLP; the effective model dimension after projection is $d = 1$, matching the stated architecture. Set $\text{PlayerEmbed}(p) = (\beta_p^o, \beta_p^d) \in \mathbb{R}^2$ for each player p, encoding both offensive and defensive scalars. Set $\text{RoleEmbed}(0) = (1, 0)$ and $\text{RoleEmbed}(1) = (0, 1)$ as one-hot role selectors, also in $\mathbb{R}^2$. The concatenated input to the projection MLP is $[\text{PlayerEmbed}(p); \text{RoleEmbed}(r)] \in \mathbb{R}^4$.

Projection MLP. For any fixed RAPM model (with specific coefficients $\{\beta_p^o, \beta_p^d\}_{p=1}^N$), the projection MLP faces exactly $2N$ required input-output pairs: $[\text{PlayerEmbed}(p); \text{RoleEmbed}(0)] \mapsto \beta_p^o$ and $[\text{PlayerEmbed}(p); \text{RoleEmbed}(1)] \mapsto \beta_p^d$ for each player p. We choose PlayerEmbed values to be distinct vectors in $\mathbb{R}^2$, making all $2N$ input points distinct elements of $\mathbb{R}^4$. By standard interpolation theory for two-layer ReLU networks, any finite set of distinct input-output pairs can be interpolated exactly by a network with sufficient width. This gives $\mathbf{z}_j^{(0)} = \beta_j^o$ for each offensive token and $\mathbf{z}_j^{(0)} = \beta_j^d$ for each defensive token.

Prediction head. With $L = 0$ and mean pooling, the lineup embedding is:

$$\mathbf{z}_{\text{lineup}} = \frac{1}{10}\Big(\sum_{j \in \mathbf{o}_i} \beta_j^o + \sum_{k \in \mathbf{x}_i} \beta_k^d\Big).$$

Set the prediction head to a linear map with weight 10 and no bias. Then:

$$\hat{y}_i \;=\; \sum_{j \in \mathbf{o}_i} \beta_j^o + \sum_{k \in \mathbf{x}_i} \beta_k^d.$$

This is exactly the RAPM prediction rule, using the architecture as specified (mean pooling, MLP projection, linear head). □ □

This establishes backward compatibility: Deep RAPM generalizes RAPM rather than replacing it, with RAPM as the $L = 0$, scalar-valued boundary case. The ℓ_2 embedding regularization reduces to the classical ridge penalty at this boundary (Proposition 5.4).

5.2 Strict Expressivity Advantage

Let $\mathcal{F}_{\text{RAPM}}$ denote the class of functions representable by classical RAPM:

$$\mathcal{F}_{\text{RAPM}} = \Big\{ f(\mathbf{o}, \mathbf{x}) = \sum_{j \in \mathbf{o}} \beta_j^o + \sum_{k \in \mathbf{x}} \beta_k^d \;\Big|\; \beta^o, \beta^d \in \mathbb{R}^N \Big\}.$$

Let $\mathcal{F}_{\text{Deep}}$ denote the class of functions representable by Deep RAPM with $d \geq 1$ and the architecture as described in Section 4.

Theorem 5.2 (Strict generalization)**.** $\mathcal{F}_{\text{RAPM}} \subsetneq \overline{\mathcal{F}_{\text{Deep}}}$*, where* $\overline{\mathcal{F}_{\text{Deep}}}$ *denotes the closure of the Deep RAPM representable class under uniform approximation on finite lineup domains.*

Proof. The inclusion $\mathcal{F}_{\text{RAPM}} \subseteq \mathcal{F}_{\text{Deep}}$ follows from Theorem 5.1. It remains to exhibit a function $f^* \in \overline{\mathcal{F}_{\text{Deep}}} \setminus \mathcal{F}_{\text{RAPM}}$ via an explicit construction.

Defining f^*. Let $N \geq 7$ and fix distinct players A, B, C, D, E, F, G. Define the pairwise co-presence indicator:

$$f^*(\mathbf{o}, \mathbf{x}) = \mathbf{1}[A \in \mathbf{o}] \cdot \mathbf{1}[B \in \mathbf{o}].$$

Step 1: $f^* \notin \mathcal{F}_{\text{RAPM}}$. Suppose for contradiction that $f^*(\mathbf{o}, \mathbf{x}) = \sum_{j \in \mathbf{o}} \beta_j^o + \sum_{k \in \mathbf{x}} \beta_k^d$ for some coefficients. Since f^* does not depend on $\mathbf{x}$, the defensive terms are a constant absorbed into an intercept; it suffices to consider $f^*(\mathbf{o}) = \sum_{j \in \mathbf{o}} \beta_j^o$.

For any additive function, the marginal exchange $f(S \cup \{a\}) - f(S \cup \{b\}) = \beta_a^o - \beta_b^o$ must be *constant* across all 4-player sets S not containing a or b. Evaluate this for players B and F across two contexts:

- $S_1 = \{A, C, D, E\}$ (contains A): $\quad f^*(S_1 \cup \{B\}) - f^*(S_1 \cup \{F\}) = 1 - 0 = 1.$
- $S_2 = \{C, D, E, G\}$ (contains neither A nor B): $\quad f^*(S_2 \cup \{B\}) - f^*(S_2 \cup \{F\}) = 0 - 0 = 0.$

Both must equal $\beta_B^o - \beta_F^o$, yielding $1 = 0$, a contradiction.

Step 2: $f^* \in \overline{\mathcal{F}_{\text{Deep}}}$ (constructive). Set $d = 1$ and $L = 0$ (no attention; this is the Deep Sets setting). Assign offensive player embeddings:

$$u_A^o = u_B^o = v \in \mathbb{R}, \quad v > 0; \qquad u_j^o = 0 \text{ for all } j \notin \{A, B\}.$$

Set all defensive embeddings to zero. After mean pooling, the offensive component of the lineup embedding is:

$$z^o(\mathbf{o}) = \frac{1}{10} \sum_{j \in \mathbf{o}} u_j^o = \begin{cases} \frac{2v}{10} & \text{if } A \in \mathbf{o} \text{ and } B \in \mathbf{o}, \\ \frac{v}{10} & \text{if exactly one of } A, B \in \mathbf{o}, \\ 0 & \text{if neither } A \text{ nor } B \in \mathbf{o}. \end{cases}$$

These three cases produce three distinct pooled values: 0, $v/10$, and $2v/10$. Since these are finitely many distinct points, a sigmoid prediction head $\hat{y} = \sigma\big(\alpha(z^o - \tau)\big)$ with slope $\alpha \to \infty$ and threshold $\tau = 3v/20$ approximates f^* to arbitrary precision over the finite support of observable lineups. That is, for any $\varepsilon > 0$ there exists a finite α such that $|\hat{y} - f^*(\mathbf{o})| < \varepsilon$ for all $\mathbf{o}$. Therefore $f^* \in \overline{\mathcal{F}_{\text{Deep}}}$, the closure of the representable class under uniform approximation. □ □

The exchange argument reveals the fundamental limitation of additive models: the marginal value of adding player B is fixed at β_B^o regardless of whether A is present. Deep RAPM's pooled embedding encodes the count of specific co-occurring players, enabling context-dependent marginal values. Note that the construction uses $L = 0$ (the Deep Sets subcase, with no attention); attention layers with $L \geq 1$ expand the representable class further to higher-order intra-lineup and cross-team interactions.

5.3 Permutation Invariance

Theorem 5.3 (Permutation invariance)**.** *Suppose that the role annotation r_j and position annotation s_j are player-attached covariates that travel with each player under any reordering (i.e., they are properties of the player, not of the player's slot index in the lineup). Then for any permutations $\pi \in S_5$ and $\sigma \in S_5$ of the offensive and defensive slots, respectively:*

$$f_\theta\big(\pi(\mathbf{o}_i),\, \sigma(\mathbf{x}_i),\, \mathbf{g}_i\big) \;=\; f_\theta\big(\mathbf{o}_i,\, \mathbf{x}_i,\, \mathbf{g}_i\big).$$

Proof. We verify invariance at each stage of the forward pass.

Input tokens. Each token $\mathbf{z}_j^{(0)} = \mathrm{MLP}([\mathrm{PlayerEmbed}(p_j); \mathrm{RoleEmbed}(r_j)])$ depends on the player's identity p_j and role r_j, not on the player's index within the lineup ordering. Permuting the offensive players by π permutes the corresponding rows of $\mathbf{Z}$; similarly for defensive players under σ.

Attention bias. The role-position index $c_j = r_j \cdot K + s_j$ is a property of the player (their basketball position s_j), not of their slot in the lineup sequence. When offensive players are permuted, their c_j values are permuted accordingly, and the bias matrix $\boldsymbol{B}_{h,c_i,c_j}$ is permuted simultaneously in both indices.

Self-attention. Multihead self-attention is permutation-equivariant: if P is the permutation matrix corresponding to any reordering of the ten rows of $\mathbf{Z}$, then $\mathrm{MultiHead}(P\mathbf{Z}) = P \cdot \mathrm{MultiHead}(\mathbf{Z})$. To see this, note that $\mathbf{Q} = (P\mathbf{Z})\mathbf{W}^Q = P(\mathbf{Z}\mathbf{W}^Q)$ and $\mathbf{K} = P(\mathbf{Z}\mathbf{W}^K)$, so $\mathbf{Q}\mathbf{K}^\top = P(\mathbf{Z}\mathbf{W}^Q\mathbf{W}^{K\top}\mathbf{Z}^\top)P^\top$, which permutes rows and columns of the attention score matrix consistently; similarly for the role-position bias. The softmax and value aggregation then produce output rows permuted by P. Each transformer layer preserves equivariance by the same argument.

Mean pooling. Mean pooling is invariant to any permutation of the input rows:

$$\frac{1}{10}\sum_{i=1}^{10}(P\mathbf{Z})_i \;=\; \frac{1}{10}\sum_{i=1}^{10}\mathbf{Z}_i.$$

Since the prediction head is applied to the pooled vector and the fixed gamestate $\mathbf{g}_i$, the final output is invariant to any permutation of the offensive lineup and any permutation of the defensive lineup. □ □

Implementation note. The theorem's hypothesis — that role and position annotations are player-attached — is essential. In practice, position labels may be derived from roster listings, heuristic assignment, or lineup slot ordering, any of which could introduce an ordering dependence. The guarantee holds precisely when r_j and s_j are queried from stable player metadata rather than inferred from the player's position in the lineup array.

5.4 Regularization and Ridge Equivalence

Proposition 5.4 (Ridge equivalence)**.** *Under the RAPM parameterization of Theorem 5.1, the Deep RAPM embedding regularization reduces exactly to the classical RAPM ridge penalty:*

$$\mathcal{L}_{\mathrm{reg}} = \lambda \sum_j \big(\|u_j^o\|^2 + \|u_j^d\|^2\big) = \lambda \sum_j \big((\beta_j^o)^2 + (\beta_j^d)^2\big).$$

Proof. When $d = 1$, $u_j^o = \beta_j^o \in \mathbb{R}$ and $u_j^d = \beta_j^d \in \mathbb{R}$, so $\|u_j^o\|^2 = (\beta_j^o)^2$ and $\|u_j^d\|^2 = (\beta_j^d)^2$. The sum over players recovers the classical ridge penalty with coefficient λ. □ □

Ridge regularization in RAPM was motivated by the sparsity of individual player observations: players appearing in few possessions have high-variance estimates, and the ridge penalty reduces this variance at the cost of shrinkage toward zero. The same motivation applies in Deep RAPM, and the ℓ_2 penalty preserves the same shrinkage intuition: embeddings for rarely-observed players are pulled toward the zero vector (the embedding analog of the average player). However, classical ridge regression's statistical stability guarantees — convexity, closed-form solutions, uniform convergence rates — do not transfer automatically to the full nonlinear model. Deep RAPM is nonconvex and highly parameterized; the penalty is a structural analog, not an inherited theorem.

5.5 Identifiability

Player embeddings in Deep RAPM are not uniquely identified from data, a property that mirrors well-known issues in linear RAPM. We describe two sources of non-uniqueness and note that their scope is more limited in the full nonlinear model than in linear factor models.

Translational non-identifiability (additive boundary). In the linear, $L = 0$ boundary case (classical RAPM), adding a constant $\delta \in \mathbb{R}$ to all offensive scalars and subtracting δ from the intercept leaves all predictions unchanged. This is the familiar RAPM collinearity: offensive and defensive ratings are not separately identified without a normalization constraint.

In the full nonlinear model, this exact translation invariance does not generally hold: the projection MLP, layer normalization, and residual connections in each transformer layer are not translation-equivariant in the embedding space. Nevertheless, gradient descent may converge to solutions with poorly centered embeddings, making individual embeddings difficult to interpret.

Fix: Impose mean-centering constraints $\sum_j u_j^o = \mathbf{0}$ and $\sum_j u_j^d = \mathbf{0}$ by subtracting the embedding mean after each gradient update, or anchor a reference player to a fixed embedding. In the $d = 1$ case this recovers the standard RAPM sum-to-zero normalization.

Representational non-uniqueness. For $d > 1$, the learned embedding space is subject to rotational and scaling indeterminacies of the type common to all deep representation learning: two models with different embedding matrices may produce identical predictions if their downstream parameters differ accordingly. Unlike a purely linear model, the exact equivalence class depends on the nonlinearities and normalizations in the network and is not simply the group of invertible linear transformations.

Implication: Individual embedding dimensions do not have intrinsic meaning and should not be interpreted literally. Embedding comparisons (player similarity, PCA visualization) are meaningful within a fixed trained model but are not directly comparable across models trained with different initializations or hyperparameters.

Fix: For visualization and interpretability, apply a post-hoc rotation to a canonical orientation (e.g., PCA axes of the offensive or defensive embedding matrix). For cross-model comparison, use prediction-based metrics (e.g., counterfactual lineup performance) rather than embedding coordinates.

In practice, mean-centering combined with ℓ_2 regularization is sufficient to ensure stable training and consistent player rankings within a single fitted model.

6 Training

6.1 Loss Functions

We support two prediction targets.

Ordinal scoring outcomes. For possession-level point totals $y \in \{0, 1, 2, 3, 4\}$, we use cross-entropy loss over five classes:

$$\mathcal{L}_{\text{CE}} = -\sum_{c=0}^{4} \mathbf{1}[y = c] \log \hat{p}_c,$$

where $\hat{p}_c = \text{softmax}(\text{MLP}_{\text{head}}(\cdot))_c$. The expected points per possession is recovered as $\sum_c c \cdot \hat{p}_c$.

Probabilistic regression. For a real-valued outcome (e.g., net rating over a stint), the head outputs a mean $\hat{\mu}$ and log-variance $\log \hat{\sigma}^2$, and we minimize Gaussian negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \log \hat{\sigma}^2 + \frac{(y - \hat{\mu})^2}{2\hat{\sigma}^2}.$$

6.2 Regularization

Player embeddings are the primary source of overfitting: a player appearing in few possessions will have a poorly identified embedding. We apply ℓ_2 regularization to the player embedding table, analogous to the ridge penalty in classical RAPM:

$$\mathcal{L}_{\text{reg}} = \lambda \sum_{j} \big(\|u_j^o\|^2 + \|u_j^d\|^2 \big).$$

The total training objective is $\mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{reg}}$.

Dropout is applied within the transformer MLP and the prediction head. The role-position bias $\boldsymbol{B}$ is not regularized, as it has $10 \times 10 \times H$ parameters and is shared across all possessions.

7 Counterfactual Lineup Evaluation

One of the most practically valuable properties of the embedding framework is the ability to evaluate *counterfactual lineups* — lineups that did not appear in the training data but whose performance we wish to predict.

7.1 The Substitution Counterfactual

For a given possession context $(\mathbf{o}_i, \mathbf{x}_i, \mathbf{g}_i)$ and a candidate player ℓ, define the *single-slot counterfactual prediction* obtained by substituting ℓ into slot m of the offensive lineup:

$$\hat{y}_i^{(\ell,m)} = f_\theta\big(\mathbf{o}_i^{-m} \cup \{\ell\},\ \mathbf{x}_i,\ \mathbf{g}_i\big),$$

where $\mathbf{o}_i^{-m}$ denotes the 4-player tuple obtained by removing slot m from $\mathbf{o}_i$. The attention mechanism recontextualizes all remaining tokens in the presence of ℓ, so the prediction accounts for synergy between ℓ and the four retained teammates as well as the full defensive lineup.

7.2 Formal Player-Value Estimand

A player-level rating — the analog of a RAPM coefficient — is obtained by averaging the substitution counterfactual over a reference distribution of lineup contexts. Fix a reference player ℓ_0 (e.g., a league-average replacement-level player). Define the *Deep RAPM offensive value* of player ℓ as:

$$V_\ell^o \;=\; \mathbb{E}_{(S,\mathbf{x},\mathbf{g})\sim\mathcal{D}_{-\ell}}\Big[f_\theta\big(S\cup\{\ell\},\,\mathbf{x},\,\mathbf{g}\big) - f_\theta\big(S\cup\{\ell_0\},\,\mathbf{x},\,\mathbf{g}\big)\Big],$$

where the expectation is over the distribution of 4-player offensive subsets S, defensive lineups $\mathbf{x}$, and gamestates $\mathbf{g}$ observed in the data (marginalized over the slot being substituted). A defensive value V_ℓ^d is defined analogously by substituting into the defensive lineup.

This estimand is a well-defined functional of the fitted model f_θ and the data distribution $\mathcal{D}$. Importantly, V_ℓ^o is *not* a context-free player invariant: it depends on the lineup distribution, the opponent distribution, and the gamestate distribution in $\mathcal{D}$. A player evaluated against a particular distribution of opponents will receive a different value than the same player evaluated against a different opponent mix; this is a feature, not a limitation, reflecting the context-dependence that the model is designed to capture. When reporting V_ℓ^o in practice, the reference distribution $\mathcal{D}$ should be specified.

Its relationship to the classical RAPM coefficient is explicit:

Proposition 7.1 (RAPM coefficient as a special case)**.** *Under the $L = 0$, $d = 1$ parameterization of Theorem 5.1, the Deep RAPM player value reduces to:*

$$V_\ell^o = \beta_\ell^o - \beta_{\ell_0}^o,$$

the classical RAPM offensive coefficient relative to the reference player.

Proof. In the linear RAPM limit, $f_\theta(S\cup\{\ell\},\mathbf{x},\mathbf{g}) - f_\theta(S\cup\{\ell_0\},\mathbf{x},\mathbf{g}) = \beta_\ell^o - \beta_{\ell_0}^o$ for every $(S,\mathbf{x},\mathbf{g})$, since the model is additive and both the reference player and the candidate differ only in the substituted slot. Taking the expectation gives the result. □ □

For $d > 1$ and $L \geq 1$, V_ℓ^o depends on the joint distribution of lineup and gamestate contexts: the same player can have different values against different opponent distributions or in different gamestate regimes. This context-dependence is the key advantage over a single scalar coefficient.

7.3 Uncertainty Quantification for Player-Value Estimates

Plug-in estimator. We distinguish two sample sizes: n_{train} possessions used to fit the model parameters $\hat\theta$, and m evaluation contexts $(Z_1,\ldots,Z_m)$ with $Z_i = (S_i,\mathbf{x}_i,\mathbf{g}_i) \sim \mathcal{D}_{-\ell}$ i.i.d. (held out from training). The natural plug-in estimator of V_ℓ^o is:

$$\hat V_\ell^o = \frac{1}{m}\sum_{i=1}^{m}\psi_\ell(Z_i;\hat\theta),$$

where $\psi_\ell(Z;\theta) = f_\theta(S\cup\{\ell\},\,\mathbf{x},\,\mathbf{g}) - f_\theta(S\cup\{\ell_0\},\,\mathbf{x},\,\mathbf{g})$ is the context-level substitution contrast, and $\hat\theta$ is the fitted Deep RAPM parameter vector. In the RAPM limit, $\psi_\ell(Z;\theta) = \beta_\ell^o - \beta_{\ell_0}^o$ is constant in Z, so the empirical average collapses to a single-value estimator, as expected.

Variance decomposition. The sampling variance of $\hat{V}_\ell^o$ decomposes into two components:

$$\mathrm{Var}(\hat{V}_\ell^o) \;=\; \underbrace{\mathbb{E}\big[\mathrm{Var}(\hat{V}_\ell^o \mid \hat{\theta})\big]}_{\text{context-sampling variance}} \;+\; \underbrace{\mathrm{Var}\big(\mathbb{E}[\hat{V}_\ell^o \mid \hat{\theta}]\big)}_{\text{parameter uncertainty}}.$$

The first term reflects averaging over a finite sample of m evaluation contexts and decreases as m grows. The second reflects uncertainty in $\hat{\theta}$ from the n_{train} training possessions.

Proposition 7.2 (Asymptotic normality of player-value estimator)**.** *Assume: (i) evaluation contexts are held out from model training; (ii)* $\sqrt{n_{\text{train}}}(\hat{\theta} - \theta_0) \to_d N(0, \Sigma_\theta)$*; (iii)* f_θ *is continuously differentiable in* θ*; (iv)* $\psi_\ell(Z; \theta_0)$ *has finite variance over* $\mathcal{D}_{-\ell}$*; (v) contexts* Z_i *are i.i.d.; and (vi)* $m/n_{\text{train}} \to \rho \in (0, \infty)$*. Then:*

$$\sqrt{m}\,(\hat{V}_\ell^o - V_\ell^o) \;\xrightarrow{d}\; N(0,\, \sigma_\ell^2),$$

with asymptotic variance

$$\sigma_\ell^2 \;=\; \underbrace{\mathrm{Var}\big(\psi_\ell(Z; \theta_0)\big)}_{\textit{context-sampling}} \;+\; \underbrace{\rho \cdot \nabla_\theta V_\ell^o(\theta_0)^\top \, \Sigma_\theta \, \nabla_\theta V_\ell^o(\theta_0)}_{\textit{parameter estimation}}.$$

Proof. Write $\hat{V}_\ell^o = m^{-1} \sum_i \psi_\ell(Z_i; \hat{\theta})$. By a first-order Taylor expansion around θ_0:

$$\sqrt{m}\,(\hat{V}_\ell^o - V_\ell^o) \;=\; \frac{1}{\sqrt{m}} \sum_i \big(\psi_\ell(Z_i; \theta_0) - V_\ell^o\big) \;+\; \nabla_\theta V_\ell^o(\theta_0)^\top \cdot \sqrt{m}(\hat{\theta} - \theta_0) \;+\; o_p(1).$$

The first term converges to $N(0, \mathrm{Var}(\psi_\ell(Z; \theta_0)))$ by the CLT. Since $\sqrt{m}(\hat{\theta} - \theta_0) = \sqrt{m/n_{\text{train}}} \cdot \sqrt{n_{\text{train}}}(\hat{\theta} - \theta_0) \to \sqrt{\rho} \cdot N(0, \Sigma_\theta)$ by assumption (ii) and $m/n_{\text{train}} \to \rho$, the second term converges to $N(0, \rho \cdot \nabla_\theta V_\ell^o(\theta_0)^\top \Sigma_\theta \nabla_\theta V_\ell^o(\theta_0))$. The two terms are asymptotically uncorrelated by assumption (i) (sample independence). Adding variances gives the result. □ □

The gradient $\nabla_\theta V_\ell^o(\theta_0)$ is computed by differentiating the plug-in average through f_θ with respect to all model parameters, which is available from automatic differentiation. This gives a delta-method standard error for $\hat{V}_\ell^o$ at the cost of one backward pass per player.

Comparison with RAPM uncertainty. In classical RAPM, the player-value estimator is $\hat{\beta}_\ell^o = [(X^\top X + \lambda I)^{-1} X^\top y]_\ell$, a linear function of the outcome vector. Under a homoskedastic linear-noise model ($y = X\beta + \varepsilon$, $\varepsilon \sim N(0, \sigma_y^2 I)$), its variance takes the closed form $\mathrm{Var}(\hat{\beta}_\ell^o) = \sigma_y^2 [(X^\top X + \lambda I)^{-1} X^\top X (X^\top X + \lambda I)^{-1}]_{\ell\ell}$. For Deep RAPM, neither term of σ_ℓ^2 is available analytically, but both are estimable: the context-sampling variance by $m^{-2} \sum_i (\psi_\ell(Z_i; \hat{\theta}) - \hat{V}_\ell^o)^2$, and the parameter-estimation term by the delta method or by bootstrap.

Bootstrap standard errors. The simplest practical uncertainty estimator is the nonparametric bootstrap. Two variants target different quantities: resampling evaluation contexts with $\hat{\theta}$ held fixed estimates the *conditional* uncertainty given the fitted model (the context-sampling component only); refitting the full model on each bootstrap training sample and re-evaluating estimates *unconditional* total variance including model-fit variability. These two procedures are not interchangeable — the first is cheaper but understates total uncertainty when n_{train} is small. For inference on individual players, bootstrap confidence intervals of the form $[\hat{V}_\ell^o \pm z_{\alpha/2} \hat{\sigma}_\ell / \sqrt{m}]$ are the recommended procedure.

7.4 Support-Sensitive Predictive Uncertainty

The variance decomposition above is global. For a specific counterfactual context Z, prediction uncertainty also depends on how well the training distribution covers the player-lineup combination being evaluated. We formalize this via a Lipschitz bound.

Support deficit. For player ℓ and context $Z = (S, \mathbf{x}, \mathbf{g})$, define the *support deficit* as the number of co-occurring players with whom ℓ has limited training co-appearances:

$$k_\ell(Z) = \big|\{j \in S \cup \mathbf{x} : n_{\text{train}}(\ell,\, j) < n_{\min}\}\big|,$$

where $n_{\text{train}}(\ell, j)$ is the number of training possessions in which players ℓ and j appeared together, and $n_{\min}$ is a minimum coverage threshold. A lineup context with $k_\ell(Z) = 0$ is fully supported; a context with $k_\ell(Z) = 9$ is entirely out of support for player ℓ.

Proposition 7.3 (Support-sensitive upper bound)**.** *Assume f_θ is locally Lipschitz with constant L with respect to each player's embedding over the region visited by the training data: for any player j and embedding values u, u' near $\hat{u}_j$,*

$$\big|f_\theta(\ldots, u_j = u, \ldots) - f_\theta(\ldots, u_j = u', \ldots)\big| \le L\|u - u'\|.$$

Let $\hat{\sigma}_j^2 = \operatorname{tr}(\hat{\Sigma}_j)$ denote a working covariance proxy for player j's embedding uncertainty (e.g., estimated from Laplace approximation, bootstrap, or the prior $\lambda^{-1} I_d$), with $\hat{\sigma}_j^2 \le \sigma_{\max}^2$ uniformly. Then, under a first-order delta-method approximation:

$$\operatorname{Var}\big(f_{\hat{\theta}}(S \cup \{\ell\},\, \mathbf{x},\, \mathbf{g})\big) \;\lesssim\; L^2 \sum_{j \in \mathcal{P}(Z)} \hat{\sigma}_j^2 \;\le\; L^2 \cdot |\mathcal{P}(Z)| \cdot \sigma_{\max}^2,$$

where $\mathcal{P}(Z) = \{\ell\} \cup S \cup \mathbf{x}$ is the set of all players in the counterfactual context. Separating players by support status (well-supported: $\hat{\sigma}_j^2 \le \sigma_{\text{low}}^2$; poorly-supported: $\hat{\sigma}_j^2 \le \sigma_{\text{high}}^2$ with $\sigma_{\text{high}}^2 \gg \sigma_{\text{low}}^2$), the bound becomes:

$$\operatorname{Var}\big(f_{\hat{\theta}}(S \cup \{\ell\},\, \mathbf{x},\, \mathbf{g})\big) \;\lesssim\; L^2\Big[(|\mathcal{P}(Z)| - k_\ell(Z))\, \sigma_{\text{low}}^2 \;+\; k_\ell(Z)\, \sigma_{\text{high}}^2\Big],$$

and this upper bound increases monotonically with the support deficit $k_\ell(Z)$.

Proof. By the delta method applied to the embedding vector $\hat{u} = (\hat{u}_j)_{j \in \mathcal{P}(Z)}$:

$$\operatorname{Var}(f_{\hat{\theta}}(Z)) \approx \nabla_u f_\theta(Z)^\top \hat{\Sigma}_u \nabla_u f_\theta(Z),$$

where $\hat{\Sigma}_u$ is the block-diagonal working covariance with block $\hat{\Sigma}_j$ for each player j. Under the local Lipschitz assumption, $\|\nabla_{u_j} f_\theta\| \le L$ for each j near the training support. For each block, Cauchy-Schwarz gives $\nabla_{u_j}^\top \hat{\Sigma}_j \nabla_{u_j} \le \|\nabla_{u_j}\|^2 \|\hat{\Sigma}_j\|_{\text{op}} \le L^2 \|\hat{\Sigma}_j\|_{\text{op}}$; since $\hat{\Sigma}_j$ is positive semidefinite, its operator norm (largest eigenvalue) is bounded above by its trace, giving $L^2 \|\hat{\Sigma}_j\|_{\text{op}} \le L^2 \operatorname{tr}(\hat{\Sigma}_j)$. Summing over players:

$$\operatorname{Var}(f_{\hat{\theta}}(Z)) \le \sum_{j \in \mathcal{P}(Z)} L^2 \operatorname{tr}(\hat{\Sigma}_j) = L^2 \sum_{j \in \mathcal{P}(Z)} \hat{\sigma}_j^2.$$

Partitioning by support status gives the second inequality. Monotonicity in $k_\ell(Z)$ holds because each additional poorly-supported player replaces a σ_{low}^2 term with $\sigma_{\text{high}}^2 > \sigma_{\text{low}}^2$, strictly increasing the upper bound. □ □

The local Lipschitz constant L can be estimated empirically as the maximum of $\|\nabla_{u_j} f_{\hat{\theta}}\|$ over players j in the training set. The working covariance proxies $\hat{\Sigma}_j$ may be obtained from the curvature of the regularized loss at $\hat{\theta}$ (Laplace approximation), from an ensemble or bootstrap, or approximated as $\lambda^{-1} I_d$ (the prior variance under ℓ_2 regularization with strength λ). The practical implication is concrete: a player value estimate $\hat{V}_\ell^o$ that aggregates many out-of-support contexts should be accompanied by a wider confidence interval, automatically widened by the support-deficit score.

7.5 Scope of Counterfactual Predictions

These counterfactual predictions are *model-based extrapolations* under the learned observational distribution. They are not causal interventional quantities. Player substitution in practice is heavily confounded: coaches substitute strategically based on score margin, matchup considerations, foul trouble, and opponent tendencies. Players who appear together in training data do so because a coach chose that lineup — a selection process that the model does not adjust for.

The quantity $\hat{y}_i^{(\ell,m)} - \hat{y}_i$ should be interpreted as the model's predictive estimate of the outcome difference if player ℓ had played in place of player $o_{i,m}$, *holding the model's learned relationships fixed*. It is an extrapolative prediction from observational data, not an estimate of the average treatment effect of a coach's substitution decision. This distinction is important: V_ℓ^o provides a principled, lineup-aware ranking of players for evaluation and roster construction, but claims of causal identification require additional structural assumptions beyond the model.

A further limitation concerns support. The model evaluates $f_\theta(S \cup \{\ell\}, \mathbf{x}, \mathbf{g})$ for lineup combinations that may have never appeared in training data. Unlike linear RAPM, which extrapolates globally through its additive structure, Deep RAPM extrapolates through its learned embeddings and attention patterns. The quality of these extrapolations depends on the coverage of similar lineup contexts in training: predictions for player ℓ inserted into a 4-player context S are better calibrated when similar (S, ℓ) combinations are well-represented in the training distribution. For players with limited court time or novel lineup roles, counterfactual estimates should be interpreted with appropriate uncertainty.

8 Interpretability

Deep RAPM trades some interpretability for expressiveness. Classical RAPM produces a single ranked list of players; Deep RAPM produces embeddings and attention patterns that require additional analysis. However, several windows into the model's learned behavior are available.

Attention weight analysis. The four quadrants of the attention matrix (Figure 1) can be inspected for each possession. By averaging attention weights across many possessions involving a given player, one can characterize which types of players (by role and position) that player most strongly attends to, and therefore which interactions are most informative for predicting their contribution.

Role-position bias matrix. The learned bias tensor $\boldsymbol{B}_{h,c_i,c_j}$ is a direct readout of the model's structural priors over role-position pairs. Visualizing $\boldsymbol{B}$ (e.g., as a 10×10 heatmap per head) reveals which category-level matchups the model has learned to amplify or suppress. If the model learns that offensive point guards strongly attend to defensive point guards (large $\boldsymbol{B}$ in the OD

quadrant for PG-vs-PG), this is evidence that primary ball-handler matchups are systematically predictive of possession outcomes.

Embedding geometry. Player embeddings u_j^o and u_j^d can be visualized in low-dimensional projections (e.g., PCA or UMAP). Clustering in this space reflects the model's learned taxonomy of offensive and defensive styles, derived purely from possession outcomes.

Limitations. Deep RAPM is less interpretable than linear RAPM at the level of individual predictions. The attention mechanism is also subject to the standard critique that attention weights are not equivalent to feature importances. We do not claim causal interpretation; the model provides a predictive and counterfactual tool grounded in outcome data.

9 Experiments

[Placeholder. The experimental evaluation is organized around three questions.]

Q1: Does Deep RAPM improve possession-level prediction? We compare Deep RAPM against: (i) linear RAPM (ridge regression baseline), (ii) a Deep Sets encoder (no attention, additive aggregation), and (iii) ablations that remove the role-position bias $\boldsymbol{B}$ or the joint 10-player encoding. Metrics: cross-entropy loss and Brier score on held-out possessions.

Q2: Does the strict expressivity advantage manifest empirically? We construct a simulation experiment in which the ground-truth outcome function contains pairwise player interactions (as in the proof of Theorem 5.2). We test whether linear RAPM fails to recover the signal while Deep RAPM succeeds, and measure the embedding dimension d required for accurate recovery.

Q3: Are player rankings meaningful? We compute V_ℓ^o and V_ℓ^d using held-out lineup contexts and compare the resulting rankings against RAPM ratings and expert consensus. Stability across seasons and against random train/test splits is reported. Counterfactual sanity checks are run against a small set of lineup combinations with known ground-truth held out from training.

10 Future Work

Two natural extensions of Deep RAPM are left to future work.

Graph Neural Network variant (DeepRAPM-GNN). Rather than dense self-attention, one can model the ten players as nodes in a graph with typed edges — intra-team (OO, DD) and inter-team (OD, DO) — and apply a graph attention network (GATv2) with message passing. This imposes a sparser interaction structure than full self-attention and may generalize better when possession data is limited. We are exploring this architecture separately.

Sequential possession modeling (DeepRAPM-RNN). The current model treats each possession as independent. A recurrent extension would encode sequences of possession-level encodings $\{[h_i^o; h_i^d; \mathbf{g}_i]\}$ through a GRU or LSTM to capture momentum, fatigue, and in-game run dynamics. This requires careful treatment of lineup substitutions and game segment boundaries. We are exploring this extension in separate work.

11 Conclusion

We introduced Deep RAPM, a Set Transformer model for possession-level player contribution in basketball. The model replaces each player's scalar RAPM coefficient with a learned latent embedding and encodes the full ten-player lineup context via multihead self-attention with learned role-position biases. The resulting 10×10 attention matrix naturally decomposes into four interpretable quadrants — offensive synergy, offensive targeting, defensive reactions, and defensive coordination — without requiring separate self-attention and cross-attention modules. Training is end-to-end with ridge-style regularization on player embeddings. Counterfactual lineup evaluation follows naturally from the embedding framework, enabling lineup-aware player value estimates that go beyond additive RAPM ratings.

References

[1] Sill, J. (2010). Improved NBA adjusted plus-minus using regularization and out-of-sample testing. *Proceedings of the MIT Sloan Sports Analytics Conference*.

[2] Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., & Smola, A. (2017). Deep sets. *Advances in Neural Information Processing Systems*, 30.

[3] Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., & Teh, Y. W. (2019). Set transformer: A framework for attention-based permutation-invariant neural networks. *Proceedings of the International Conference on Machine Learning (ICML)*.